



UNIVERSIDAD AUTÓNOMA METROPOLITANA
Unidad Azcapotzalco

División de Ciencias Básicas e Ingeniería
Licenciatura en Ingeniería en Computación



Manual de Instalación — Sistema CyAD

Proyecto Tecnológico

Manual de Instalación versión

Trimestre 2026-Primavera

[TU NOMBRE]

[MATRÍCULA]

al[MATRÍCULA]@azc.uam.mx

[Grado. Nombres Apellidos del Asesor]

[Profesor Titular/Asociado/Asistente]

[Departamento de Sistemas]

[correo@azc.uam.mx]

22 de agosto de 2026

Declaratoria

En caso de que el Comité de Estudios de la Licenciatura en Computación apruebe la realización de la presente propuesta, otorgamos nuestra autorización para su publicación en la página de la División de Ciencias Básicas e Ingeniería.

[TU NOMBRE]

[Grado. Nombres Apellidos del Asesor]

1. Introducción

Este manual describe cómo instalar y ejecutar el *Sistema para la gestión de formularios sobre los cursos y autoevaluaciones docentes* de la División de Ciencias y Artes para el Diseño (CyAD) en un entorno de desarrollo local. La instalación en producción sigue los mismos pasos, cambiando únicamente el archivo `.env` y el modo de servido (*gunicorn* + *Nginx*).

2. Prerrequisitos

Software	Versión mínima	Uso
Python	3.11+ (probado con 3.13)	Ejecución del backend Django.
Node.js	20+ (probado con 22 LTS)	Ejecución del frontend Vite + React.
PostgreSQL	14+ (probado con 17)	Base de datos relacional.
Git	2.40+	Clonado del repositorio.
<i>npm</i> (incluido con Node)	10+	Gestor de paquetes JS.

En Windows se recomienda utilizar *Git Bash* o *PowerShell*. En Linux/macOS los comandos son idénticos, cambiando la ruta del entorno virtual (`bin/` en lugar de `Scripts/`).

Nota importante para Windows: el instalador oficial de PostgreSQL no agrega `psql` al PATH del sistema. Si al escribir `psql -version` obtiene “command not found”, agregue manualmente la carpeta `C:\Program Files\PostgreSQL\17\bin` (ajuste al número de versión que tenga) a la variable de entorno `Path` de Windows, o invoque `psql` por ruta completa.

2.1. Requisitos mínimos de hardware

Las siguientes cifras se derivan de la re prueba de despliegue documentada en `hallazgos_despliegue.pdf`: el proceso de Django consume aproximadamente 120 MB de RAM en reposo, PostgreSQL alrededor de 80 MB con la carga de demostración, y el *build* del frontend (`npm run build`) alcanza un pico de ~250 MB durante la compilación (no en tiempo de ejecución).

Escenario	RAM mínima	RAM dada	recomen-	Disco
Desarrollo local (una sola persona)	4 GB	8 GB		5 GB libres

Escenario	RAM mínima	RAM dada	recomen-	Disco
Servidor pequeño (<50 profesores, un solo periodo activo)	2 GB	4 GB		10 GB libres
Servidor CyAD completo (>300 profesores, histórico multi-periodo)	4 GB	8 GB		20 GB libres

Sobre CPU: cualquier procesador x86-64 de doble núcleo publicado en la última década es suficiente para el runtime; el cuello de botella es la memoria durante `npm run build`, que sólo ocurre al desplegar una nueva versión, no en operación diaria. En un servidor dedicado bastan *dos vCPU*; para desarrollo local la máquina que edita el código suele tener recursos de sobra.

Ancho de banda: al ser una aplicación de tipo formulario, el consumo por sesión es del orden de decenas de KB por interacción. Cualquier enlace institucional (> 10 Mbps) resulta holgado.

3. Clonado del repositorio

```
git clone <URL_del_repositorio> proyecto_terminal_cyad
cd proyecto_terminal_cyad
```

4. Configuración del backend

4.1. Entorno virtual

```
cd backend
python -m venv .venv
.venv\Scripts\activate # Windows
source .venv/bin/activate # Linux/macOS
pip install -r requirements.txt
```

4.2. Base de datos

El proyecto utiliza por defecto el usuario `postgres` y una base de datos llamada `proyecto_terminal_cyad` (ver `backend/.env.example`). La forma recomendada es dejar que el script auxiliar cree la base de datos automáticamente después de configurar el archivo `.env` (siguiente sección):

```
python scripts/create_db.py
```

Si prefiere crearla manualmente con `psql`:

```
psql -U postgres
CREATE DATABASE proyecto_terminal_cyad;
```

También puede definir un usuario dedicado con permisos limitados (recomendado en despliegues compartidos) y ajustar DB_USER/DB_PASSWORD en el archivo `.env` en consecuencia:

```
CREATE USER cyad_user WITH PASSWORD 'cyad_pass';
GRANT ALL PRIVILEGES ON DATABASE proyecto_terminal_cyad TO
cyad_user;
```

4.3. Archivo `.env`

Ubicación: el archivo `.env` que Django lee debe estar en la carpeta `backend/` (donde vive `manage.py`), *no* en la raíz del repositorio. El repositorio incluye `backend/.env.example` como plantilla; cópielo y ajuste los valores:

```
cd backend
cp .env.example .env
```

Contenido esperado (valores por defecto de la plantilla; ajuste DB_PASSWORD y SECRET_KEY a su entorno):

```
DEBUG=True
SECRET_KEY=cambia-esta-clave-por-una-aleatoria-larga
ALLOWED_HOSTS=localhost,127.0.0.1
DB_NAME=proyecto_terminal_cyad
DB_USER=postgres
DB_PASSWORD=tu_password_de_postgres
DB_HOST=localhost
DB_PORT=5432
JWT_ACCESS_MINUTES=60
JWT_REFRESH_DAYS=7
CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

Nota: existe también un archivo `.env.example` en la raíz del repositorio; *no* es el que lee Django y su esquema de variables JWT (`ACCESS_TOKEN_LIFETIME`, `REFRESH_TOKEN_LIFETIME`) no coincide con lo que espera `backend/config/settings.py`. Utilice siempre `backend/.env.example`.

Para producción se debe establecer `DEBUG=False`, definir `ALLOWED_HOSTS` con el dominio real y generar una `SECRET_KEY` con `python -c "from django.core.management import get_random_secret_key; print(get_random_secret_key())"`.

4.4. Migraciones

```
python manage.py migrate
```

4.5. Bootstrap unificado (Windows PowerShell)

Como atajo, el repositorio incluye `scripts/bootstrap.ps1` que orquesta *creación de BD, migraciones y siembra de datos de demo* en un único comando. Debe ejecutarse desde la raíz del repositorio, con el `.venv` ya activo y `backend/.env` configurado:

```
.\scripts\bootstrap.ps1
```

El script imprime tres pasos numerados ([1/3] crear BD, [2/3] migrar, [3/3] seed) y al finalizar recuerda cómo arrancar el backend y el frontend en dos terminales distintas. Falla temprano si falta `backend/.env` o si el árbol del repositorio no está completo.

Para Linux o macOS, ejecute los tres pasos por separado siguiendo la siguiente sección.

4.6. Datos iniciales

La forma más rápida de dejar el sistema listo para usar es ejecutar el *seed* de demostración, que es idempotente y crea catálogos, UEAs, usuarios de prueba, periodo activo y el formulario de autoevaluación en un solo comando:

```
python scripts/seed_demo.py
```

De forma alternativa, se pueden cargar los catálogos base y los usuarios mínimos por separado (útil cuando ya existen datos):

```
python manage.py loaddata catalogos/fixtures/departamentos.json
python manage.py loaddata catalogos/fixtures/licenciaturas.json
python manage.py loaddata catalogos/fixtures/posgrados.json
python manage.py loaddata catalogos/fixtures/areas.json
python manage.py loaddata catalogos/fixtures/periodos.json
python scripts/seed_users.py
```

Al finalizar, existirán las siguientes cuentas:

Rol	Correo	Contraseña
Administrador	admin@cyad.uam.mx	Admin1234!
Profesor	profesor@cyad.uam.mx	Profesor1234!

Estas contraseñas deben cambiarse antes del despliegue a producción.

5. Instalación mínima sin datos de demo

Para un despliegue institucional real (donde el administrador cargará sus propios departamentos, licenciaturas, UEAs y profesores) no se debe ejecutar `seed_demo.py`: ese *script* crea un profesor de prueba, un formulario ya publicado y respuestas ficticias que ensucian la base de datos productiva. La ruta correcta es:

1. **Crear la base de datos vacía** (con `scripts/create_db.py` o manualmente con `psql`, según la sección anterior).

2. **Aplicar migraciones:**

```
python manage.py migrate
```

3. **Crear la primera cuenta de administrador** con el comando interactivo de Django:

```
python manage.py createsuperuser
```

Se pedirán correo, nombre y contraseña. Este usuario tendrá rol ADMIN y podrá crear el resto de cuentas desde la interfaz web.

4. **Cargar los catálogos institucionales base** desde los *fixtures* versionados en el repositorio:

```
python manage.py loaddata catalogos/fixtures/departamentos.json
python manage.py loaddata catalogos/fixtures/licenciaturas.json
python manage.py loaddata catalogos/fixtures/posgrados.json
python manage.py loaddata catalogos/fixtures/areas.json
python manage.py loaddata catalogos/fixtures/periodos.json
```

Estos JSON contienen la estructura oficial de CyAD (nueve departamentos, tres licenciaturas, posgrados vigentes y áreas por licenciatura) y son idempotentes: pueden re-ejecutarse sin duplicar registros.

5. **Cargar las UEAs** desde el CSV maestro que mantiene coordinación académica (formato: clave, nombre, licenciatura, tipo, créditos, etc.):

```
python manage.py cargar_catalogos_csv <ruta/al/uea.csv>
```

El comando valida claves duplicadas y reporta las filas rechazadas. Si aún no se dispone del CSV, las UEAs pueden crearse una por una desde Catálogos → UEA en la interfaz.

6. **Iniciar sesión** con el superuser creado en el paso 3 y, desde el constructor de formularios, generar el primer formulario de autoevaluación institucional.

Advertencia: no ejecute `python scripts/seed_demo.py` en un ambiente productivo. Está diseñado exclusivamente para el flujo de desarrollo local y crea datos ficticios que después será tedioso eliminar. El *script* `scripts/bootstrap.psql` descrito arriba tampoco es apropiado para producción porque incluye el paso de *seed*; en producción se corre cada paso por separado como se muestra en esta sección.

Los usuarios de profesor se crean posteriormente desde Profesores → Nuevo profesor en la interfaz de administrador; cada creación genera un correo + contraseña temporal que el administrador entrega personalmente al docente.

5.1. Servidor de desarrollo del backend

```
python manage.py runserver 8000
```

El backend queda disponible en `http://localhost:8000`. La documentación interactiva de la API está en `http://localhost:8000/api/docs/`.

6. Configuración del frontend

En una terminal aparte:

```
cd frontend
npm install
npm run dev
```

El frontend queda disponible en `http://localhost:5173`. Vite realiza *hot reload* automático al editar archivos fuente.

Si el backend no está en `localhost:8000`, cree `frontend/.env.local` con:

```
VITE_API_URL=http://tu-servidor:8000/api/v1
```

Nota: el nombre correcto de la variable es `VITE_API_URL` (verificable en `frontend/src/api/client.js`).

7. Verificación

1. Abra `http://localhost:5173/` — debe mostrarse la vista pública “Explorar”.
2. Inicie sesión con `admin@cyad.uam.mx`. Debe redirigir al *dashboard* de administrador.
3. Cierre sesión, inicie con `profesor@cyad.uam.mx`. Debe redirigir al *dashboard* de profesor.
4. Ejecute la suite de pruebas: `pytest` desde `backend/`. Todas deben pasar.

8. Comandos administrativos útiles

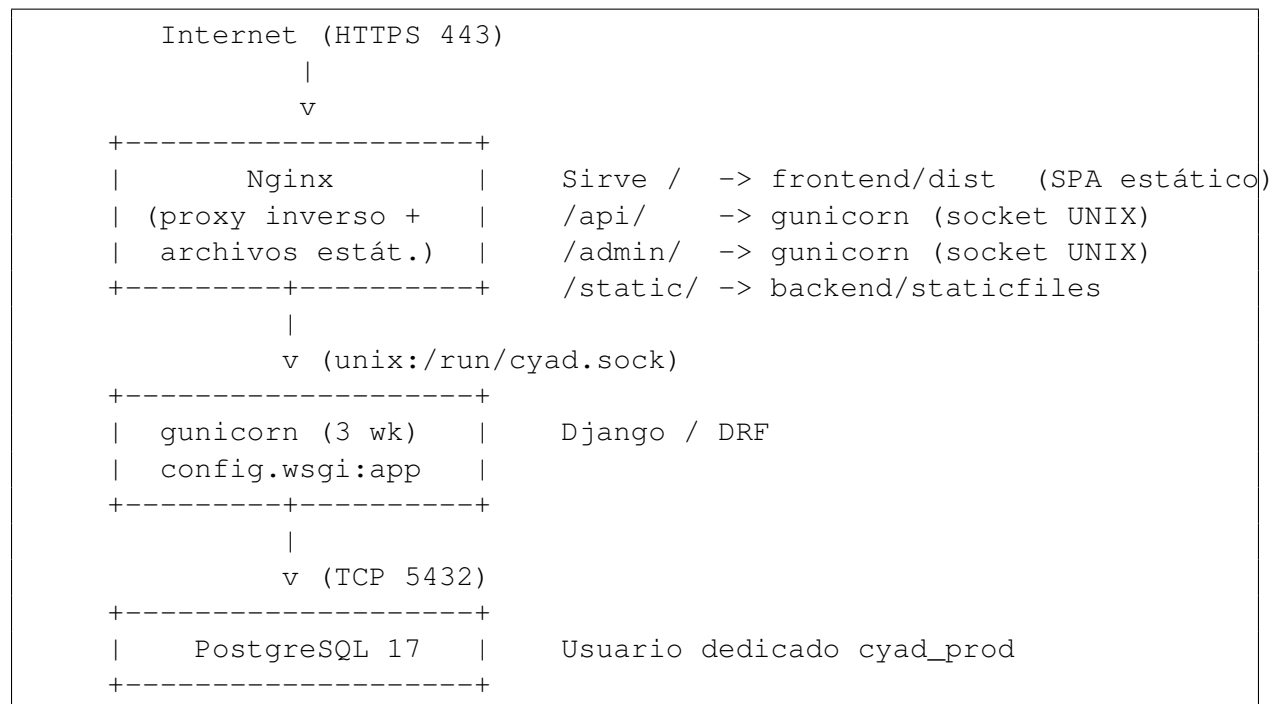
Comando	Uso
<code>python manage.py cargar_catalogos_csv <archivo></code>	Importar catálogos desde CSV.
<code>python manage.py createsuperuser</code>	Crear cuenta administradora adicional.

Comando	Uso
<code>python manage.py dumpdata <app></code>	Exportar datos a JSON.
<code>pytest -cov=. -cov-report=html</code>	Ejecutar pruebas con cobertura.
<code>npm run build (frontend)</code>	Compilar producción estática (dist/).

9. Despliegue en servidor de producción

Esta sección describe el despliegue en un servidor Linux (Ubuntu 22.04 LTS o Debian 12) que atienda las peticiones institucionales. El manual asume que ya se cumplen los prerequisites (Sección 5 inclusive: base limpia, superuser creado, catálogos cargados).

9.1. Topología recomendada



Todos los servicios corren en la misma máquina en el caso más simple; se pueden separar la BD y el *app server* sin cambios en el código, ajustando `DB_HOST` en el `.env`.

9.2. Variables de entorno de producción

El archivo `backend/.env` para producción difiere del de desarrollo en cinco campos críticos:

```

DEBUG=False
SECRET_KEY=<clave-larga-generada-abajo>
ALLOWED_HOSTS=cyad.uam.mx

```

```
CORS_ALLOWED_ORIGINS=https://cyad.uam.mx
DB_NAME=cyad_prod
DB_USER=cyad_prod
DB_PASSWORD=<password-fuerte>
DB_HOST=localhost
DB_PORT=5432
JWT_ACCESS_MINUTES=15
JWT_REFRESH_DAYS=1
```

Generación de SECRET_KEY:

```
python -c "from django.core.management.utils import get_random_
secret_key;
print(get_random_secret_key())"
```

La ventana de `JWT_ACCESS_MINUTES=15` y `JWT_REFRESH_DAYS=1` reduce el impacto de un token filtrado; el frontend refresca el *access token* automáticamente en segundo plano mientras el *refresh* siga vigente.

El archivo `.env` debe tener permisos restrictivos:

```
chmod 600 backend/.env
chown www-data:www-data backend/.env
```

9.3. Base de datos en producción

Crear un rol *no-superuser* dedicado y una base propiedad de ese rol:

```
sudo -u postgres psql
CREATE USER cyad_prod WITH PASSWORD '<password-fuerte>';
CREATE DATABASE cyad_prod OWNER cyad_prod;
\q
```

Endurecer `pg_hba.conf` para permitir sólo conexiones locales autenticadas por contraseña al usuario `cyad_prod` (no permita accesos remotos al usuario `postgres`).

Respaldo diario recomendado vía *cron*:

```
# /etc/cron.d/cyad-backup
0 3 * * * postgres pg_dump -Fc cyad_prod \
> /var/backups/cyad/cyad_$(date +%Y%m%d).dump
```

Rotar respaldos con `logrotate` o un simple `find /var/backups/cyad -mtime +30 -delete`.

9.4. Backend con gunicorn + systemd

Instalar gunicorn en el mismo entorno virtual:

```
pip install gunicorn
```

Crear la unidad `/etc/systemd/system/cyad-backend.service`:

```
[Unit]
Description=CyAD Django backend (gunicorn)
After=network.target postgresql.service
Requires=postgresql.service

[Service]
Type=notify
User=www-data
Group=www-data
WorkingDirectory=/srv/cyad/proyecto_terminal_cyad/backend
Environment="PATH=/srv/cyad/proyecto_terminal_cyad/backend/.venv/bin"
ExecStart=/srv/cyad/proyecto_terminal_cyad/backend/.venv/bin/gunicorn \
    config.wsgi:application \
    --workers 3 \
    --bind unix:/run/cyad.sock \
    --access-logfile /var/log/cyad/access.log \
    --error-logfile /var/log/cyad/error.log
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Habilitar y arrancar:

```
sudo mkdir -p /var/log/cyad
sudo chown www-data:www-data /var/log/cyad
sudo systemctl daemon-reload
sudo systemctl enable -now cyad-backend
sudo systemctl status cyad-backend
```

9.5. Archivos estáticos del backend

El Django admin y drf-spectacular sirven CSS/JS estáticos que Nginx debe entregar directamente. Recolectarlos en `backend/staticfiles/` (ruta ya definida como `STATIC_ROOT` en `config/settings.py`):

```
python manage.py collectstatic --noinput
```

Debe re-ejecutarse en cada actualización que añada estáticos nuevos.

9.6. Frontend en producción

Compilar los estáticos del SPA:

```
cd frontend
npm ci
npm run build
```

`npm ci` respeta `package-lock.json` exactamente y es más rápido que `npm install` para despliegues reproducibles. El resultado queda en `frontend/dist/`, que Nginx servirá como archivos estáticos.

Antes de compilar, crear `frontend/.env.production` con la URL pública de la API:

```
VITE_API_URL=https://cyad.uam.mx/api/v1
```

9.7. Nginx como reverse proxy

Crear `/etc/nginx/sites-available/cyad.conf`:

```
server {
    listen 80;
    server_name cyad.uam.mx;

    # SPA: sirve archivos estáticos y hace fallback a index.html
    root /srv/cyad/proyecto_terminal_cyad/frontend/dist;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }

    # Backend (API + admin) via socket UNIX
    location /api/ {
        proxy_pass http://unix:/run/cyad.sock;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /admin/ {
        proxy_pass http://unix:/run/cyad.sock;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

```
}

# Estáticos del backend (Django admin, drf-spectacular)
location /static/ {
    alias /srv/cyad/proyecto_terminal_cyad/backend/staticfiles/;
    expires 30d;
}

client_max_body_size 10M;
}
```

Habilitar y recargar:

```
sudo ln -s /etc/nginx/sites-available/cyad.conf
/etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx
```

9.8. HTTPS con Let's Encrypt

Con *certbot* instalado (`sudo apt install certbot python3-certbot-nginx`):

```
sudo certbot -nginx -d cyad.uam.mx
```

Certbot modifica el bloque de Nginx anterior para añadir escucha en :443 con el certificado emitido y una redirección 80 → 443. La renovación se instala automáticamente vía *systemd timer*; verificar con `sudo systemctl status certbot.timer`.

9.9. Actualizaciones (redeploy)

Cada nueva versión del código se despliega con:

```
cd /srv/cyad/proyecto_terminal_cyad
git pull
cd backend
source .venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py collectstatic --noinput
cd ../frontend
npm ci
npm run build
sudo systemctl restart cyad-backend
sudo systemctl reload nginx
```

Es recomendable hacer `pg_dump` antes de cada `migrate` en producción.

9.10. Nota para despliegues en Windows Server

Si el hosting institucional obliga a usar Windows Server, gunicorn no está disponible. La alternativa recomendada es *waitress*:

```
pip install waitress
waitress-serve -port=8000 config.wsgi:application
```

Como *reverse proxy* se usa IIS con el módulo *URL Rewrite + Application Request Routing*, o bien *Nginx for Windows* con una configuración equivalente a la mostrada arriba. El servicio se registra con `sc.exe create` o mediante *NSSM*. El resto de la configuración (`.env`, base de datos, `collectstatic`, `npm run build`) es idéntica.

10. Solución de problemas

- **Error FATAL: password authentication failed:** verifique `DB_USER` y `DB_PASSWORD` en `.env` y que el usuario tenga permisos sobre la base de datos.
- **Error CORS al iniciar sesión desde el frontend:** verifique que `CORS_ALLOWED_ORIGINS` incluya `http://localhost:5173`.
- **429 Too Many Requests al iniciar sesión:** se activó el *rate limiting* (5 intentos por minuto). Espere 60 segundos y reintente.
- **Puerto 5173 ocupado:** Vite usará automáticamente el siguiente puerto libre; verifique la URL impresa al inicio.
- **seed_demo.py falla con "La suma de pesos de las secciones debe ser 100 %"**: en algunas versiones del script las secciones se crean sin peso, lo que hace fallar la publicación del formulario. Como *workaround* temporal, edite `backend/scripts/seed_demo.py` y añada `peso=Decimal("50")` a la creación de cada una de las dos `Seccion.objects.create(...)`, luego ejecute `python manage.py flush --noinput` y vuelva a lanzar el seed. Ver `hallazgos_despliegue.pdf` para detalles.
- **psql: command not found en Windows:** agregue `C:\Program Files\PostgreSQL\<ver>` al PATH, o llame a `psql` por ruta completa. Alternativamente, use *pgAdmin* o el script `scripts/create_db.py`, que utiliza el driver Python y no requiere `psql` en el PATH.